

Spectral-Seeded NSGA-II for Multi-Application Mapping in Many-Core Systems

Ali Bölücü*

*Computer Engineering Department, Hacettepe University, Ankara, Türkiye

†Graduate School of Science and Engineering, Hacettepe University, Ankara, Türkiye

E-mails: *alibolucu@hacettepe.edu.tr

Abstract—Mapping multiple applications onto a Network-on-Chip (NoC) is a complex optimization problem. Evolutionary algorithms like NSGA-II are widely used for this, but their reliance on random starting points can cause slow convergence and lower-quality results. To address this, we propose a Spectral-Seeded NSGA-II approach. Using spectral graph partitioning and Fiedler vector analysis, we group tasks that communicate heavily and map them close to each other. We then use these optimized groups as starting seeds for the NSGA-II algorithm to guide the search process more effectively. Experimental results show that this method improves convergence speed, allowing the algorithm to reach good mappings faster. Specifically, the spectral-seeded approach provides an effective balance between communication cost and thermal distribution—either by finding better objective values or by converging to similar solutions in fewer iterations. Furthermore, it yields improved Inverted Generational Distance (IGD) scores compared to traditional random initialization.

Index Terms—Network-on-Chip (NoC), Application Mapping, NSGA-II, Spectral Clustering, Multiprocessor System-on-Chip (MPSoC).

I. INTRODUCTION

As Multiprocessor Systems-on-Chip (MPSoC) scale to hundreds of cores, Network-on-Chip (NoC) has become the standard communication paradigm to overcome the limitations of traditional bus-based architectures [1], [2]. However, the overall efficiency of an NoC relies heavily on application mapping: the assignment of concurrent application tasks to available processing cores. Poor task mapping leads to long communication distances, increased latency, thermal hotspots, and excessive energy consumption [3], [4].

Mapping multiple applications to an NoC is a highly complex (NP-hard) problem [3], [5]. Researchers often use Multi-Objective Evolutionary Algorithms (MOEAs), such as NSGA-II, to balance conflicting goals at the same time [6], [7]. A major weakness of standard MOEAs is that they start with a random population. In the huge search space of NoC mapping, random starting points slow down the algorithm and often cause it to get stuck in poor solutions (local optima) [2], [6]. On the other hand, graph-based methods such as spectral partitioning can quickly find good mappings based on the network structure [8]–[10]. However, they are static and cannot optimize multiple dynamic goals like MOEAs do.

To combine the strengths of both methods, this paper proposes *Spectral-Seeded NSGA-II for Multi-Application Mapping*. First, we use a pre-processing stage based on spectral graph theory to group tasks that communicate heavily and

map them close to each other on the NoC. This reduces the communication distance. We then use these smart mappings as the starting population (seeds) for the NSGA-II algorithm [1]–[5]. By combining the mathematical precision of spectral partitioning with the search power of MOEAs, our method significantly speeds up the optimization process. It also produces much better and more diverse solutions for balancing communication cost and temperature compared to traditional random-start algorithms.

II. RELATED WORK

The application mapping problem in Network-on-Chip (NoC) architectures has been extensively studied to optimize performance metrics such as communication cost, latency, and energy consumption. Existing approaches in the literature can be broadly classified into three main categories: pure meta-heuristic algorithms, pure clustering and graph partitioning methods, and hybrid approaches.

In the first category, pure metaheuristic methods such as Multi-Objective Evolutionary Algorithms (MOEAs) are widely used due to their robust global search capabilities in massive design spaces. For instance, de Almeida et al. [7] emphasized the necessity of using NSGA-II for mapping multiple concurrent applications to balance conflicting dynamic metrics like fault tolerance and thermal distribution. However, a comprehensive review by Verma et al. [6] highlights a fundamental limitation of standard MOEAs: they rely on random population initialization. In the highly complex search space of NoC mapping, this blind starting point wastes significant computational resources on evaluating poor-quality configurations, leading to slow convergence and a high risk of getting trapped in local optima.

In the second category, to avoid the slow convergence of evolutionary algorithms, some researchers rely entirely on pure structural graph partitioning and clustering heuristics. Todorov et al. [9] demonstrated that spectral clustering can effectively minimize communication cut-sizes by analyzing the eigenvalues of application task graphs. Similarly, constructive methods like SMA [10] and SpecMap [8] use deterministic graph partitioning to quickly find topologically sound mappings. While pure spectral methods are extremely fast and excel at minimizing communication distances, they are entirely static. They lack the evolutionary mechanisms required to simultaneously optimize multiple dynamic physical metrics (e.g.,

thermal hotspots and end-to-end latency) that vary depending on the physical placement.

In the third category, recent research has shifted towards hybrid approaches to combine the speed of clustering with the optimization power of metaheuristics. These methods use clustering to intelligently seed the initial population of metaheuristics. For example, Sikandar et al. [3] used K-Nearest Neighbor (KNN) clustering to accelerate the Sailfish algorithm, while Mehmood et al. [4] utilized density-based clustering for the Andean Condor Algorithm. Amin et al. [1], [2] further improved this domain by integrating K-Means with the Elbow method and Polynomial Regression to guide search areas. Recently, Li et al. [5] leveraged spectral clustering to capture task communication strength and seed the Coati Optimization Algorithm.

While these hybrid methods successfully speed up the optimization process, they predominantly target single-application scenarios and utilize single-objective cost functions. This severely limits their ability to balance conflicting metrics in modern many-core systems running multiple applications simultaneously. As summarized in Table I, existing works either suffer from random starts in multi-application scenarios [7], use static methods lacking evolutionary optimization [8]–[10], or apply hybrid methods only to single-objective problems [1]–[5].

Our proposed *Spectral-Seeded NSGA-II* framework addresses this exact gap. By calculating the Laplacian matrix of a unified multi-application affinity graph and extracting the Fiedler vector, we deterministically group heavily communicating tasks. We then use these structurally optimized groups as the starting population (seeds) for the NSGA-II algorithm. This hybrid approach perfectly bridges the mathematical precision of spectral partitioning with the multi-objective, multi-application optimization power of NSGA-II.

TABLE I
COMPARISON OF APPLICATION MAPPING METHODOLOGIES IN NoC

Work	Year	Optimization Engine	Initialization Strategy	Objective Type	App Scale	Optimized Metrics
Sikandar et al. [3]	2021	SFOA	Shared KNN	SO	Single-App	Power, Comm. Cost
Amin et al. [1]	2022	HPSA	K-Means (Elbow)	SO	Single-App	Energy, Latency, Power
Alagunany et al. [10]	2022	SMA (Static)	Spectral	SO	Single-App	Comm. Bandwidth, Power
Mehmood et al. [4]	2022	ACA	Comm. Density	SO	Single-App	Comm. Cost, Energy, Latency
Raj et al. [8]	2023	SpecMap (Static)	Spectral	SO	Multi-App	Comm. Cost, Latency, Energy
Amin et al. [2]	2023	CNOC-PR	K-Means (Silhouette)	WS	Single-App	Comm. Cost, Power, Latency
Li et al. [5]	2025	ET (COA)	Spectral (Silhouette)	SO	Single-App	Comm. Cost, Energy, Latency
de Almeida et al. [7]	2025	NSGA-II	Random	MO	Multi-App	Fault Tolerance, Thermal, Latency
This Work	2026	NSGA-II	Spectral (Fiedler) + K-Means (Silhouette)	MO	Multi-App	Comm. Cost, Thermal

SO: Single-Objective (optimizes a single cost metric), WS: Weighted-Sum (aggregates metrics into a scalar), MO: Multi-Objective (generates a Pareto-optimal front of trade-off solutions).

III. PROBLEM DEFINITION

The multi-application mapping problem in Many-Core Network-on-Chip (NoC) systems involves assigning a set of concurrent application tasks to physical processing cores. Because the number of possible mapping combinations grows factorially with the number of tasks and cores, this process is mathematically classified as an NP-hard problem [3], [5]. This section defines the high-level system context, the nature of the applications, and the primary objectives governing the mapping optimization.

A. System and Application Description

This work uses a static (design-time) mapping approach to place multiple applications onto a 2D-Mesh NoC architecture at the same time. We model these applications as Directed Acyclic Graphs (DAGs). In these graphs, nodes are tasks, and arrows (edges) show the required communication bandwidth between tasks.

Before optimization begins, we merge all application DAGs into a single *Affinity Matrix* (A) to capture the global communication load. In this matrix, an entry $A_{i,j}$ represents the bandwidth required between Task i and Task j . Tasks that do not communicate, such as those from completely different applications, simply have an entry of zero. This unified matrix allows our system to analyze the entire communication topology at once, rather than processing each application one by one. The target NoC platform uses standard XY routing to prevent data deadlocks.

Mapping multiple applications creates a massive search space. Because of this, using evolutionary algorithms with random starting points is very slow and inefficient. To solve this, our system first sends the task communication data to a Spectral Partitioning module. This module analyzes the network structure to create highly optimized initial seeds. Finally, the NSGA-II algorithm uses these smart seeds to quickly find the best possible mappings.

B. Objectives

The proposed framework solves a bi-objective optimization problem, specifically designed to address the dynamic constraints of modern NoC platforms. Unlike traditional approaches that isolate metrics, this formulation employs hybrid objectives:

- **Communication Cost (f_1):** Minimizes both the maximum worst-case communication cost (bottleneck) and the average communication cost across the network.
- **Thermal Hotspot Cost (f_2):** Prevents localized overheating by modeling physical heat dissipation between neighboring cores, aiming to minimize both the peak temperature and the overall thermal stress.

C. Constraints

To ensure physical realizability and validity of the generated mappings, the optimization is subject to the following constraints:

- **Mapping Exclusivity:** Each processing core can host at most one task (1:1 mapping constraint).
- **Complete Assignment:** Every task within the multi-application graph must be assigned to exactly one processing core.

D. Mathematical Formulation

We formally define the merged multi-application structure as a task graph $G(T, E)$ and the target 2D-Mesh architecture as $A(C, P)$.

Decision Variables: The primary decision variable is binary and determines whether a task is assigned to a specific processing core:

$$x_{i,u} = \begin{cases} 1, & \text{if task } t_i \text{ is mapped to core } c_u \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

Objective 1: Communication Cost (f_1): The communication cost $L_{i,j}$ for a specific edge $(i,j) \in E$ is defined as the product of its communication volume ($w_{i,j}$) and the Manhattan distance ($H_{u,v}$) between the assigned cores:

$$L_{i,j} = \sum_{u=1}^M \sum_{v=1}^M x_{i,u} \cdot x_{j,v} \cdot w_{i,j} \cdot H_{u,v} \quad (2)$$

To balance worst-case bottlenecks and overall network congestion, the total communication cost objective f_1 aggregates the maximum edge cost (L_{max}) and the mean edge cost (L_{mean}). Because these two components exist on fundamentally different numerical scales, they are first independently scaled to a $[0, 1]$ range using Min-Max normalization based on empirically discovered ideal and nadir bounds (where *ideal* represents the theoretical best-case minimum, and *nadir* represents the worst-case maximum):

$$L'_{max} = \frac{L_{max} - L_{max}^{ideal}}{L_{nadir}^{ideal} - L_{max}^{ideal}}, \quad L'_{mean} = \frac{L_{mean} - L_{mean}^{ideal}}{L_{nadir}^{ideal} - L_{mean}^{ideal}} \quad (3)$$

The final objective function is then computed as a weighted sum of these normalized metrics:

$$\min f_1 = w_{max} \cdot L'_{max} + w_{mean} \cdot L'_{mean} \quad (4)$$

where w_{max} and w_{mean} are tunable weighting parameters (subject to $w_{max} + w_{mean} = 1$) that dictate the priority between resolving bottleneck links and reducing general network traffic.

Objective 2: Thermal Hotspot Cost (f_2): The base heat h_i generated by a task t_i is proportional to its total incoming and outgoing communication volume:

$$h_i = \sum_{j:(i,j) \in E} w_{i,j} + \sum_{k:(k,i) \in E} w_{k,i} \quad (5)$$

The initial heat of a core c_u depends entirely on the task assigned to it:

$$Q_u = \sum_{i=1}^N x_{i,u} \cdot h_i \quad (6)$$

To accurately model physical heat dissipation, the final temperature T_u of core c_u incorporates heat transferred from its direct spatial neighbors (where distance is 1 hop) with a dissipation coefficient $\alpha = 0.3$:

$$T_u = Q_u + \alpha \sum_{v \in \text{Neighbors}(u)} Q_v \quad (7)$$

The thermal objective f_2 minimizes both the regional peak hotspot ($T_{max} = \max_u T_u$) and the average chip temperature ($T_{mean} = \frac{1}{M} \sum T_u$). Similar to the communication cost, these metrics are Min-Max normalized to T'_{max} and T'_{mean} to ensure scale invariance:

$$\min f_2 = w_{hotspot} \cdot T'_{max} + w_{mean} \cdot T'_{mean} \quad (8)$$

where $w_{hotspot}$ and w_{mean} are tunable coefficients (with $w_{hotspot} + w_{mean} = 1$) governing the trade-off between peak temperature mitigation and overall heat reduction.

Symbol	Description
$G(T, E)$	Unified multi-application task graph (DAG)
T, N	Set of tasks $T = \{t_1, t_2, \dots, t_N\}$ and total task count N
E	Set of directed edges representing task dependencies
$w_{i,j}$	Communication volume from task t_i to t_j
$A(C, P)$	NoC architecture graph with physical links P
C, M	Set of processing cores $C = \{c_1, \dots, c_M\}$ and total core count M
$H_{u,v}$	Manhattan distance (hop count) between cores c_u and c_v
$x_{i,u}$	Binary mapping decision variable
$L_{i,j}$	Communication cost for edge (i, j)
L'_{max}, L'_{mean}	Normalized max and mean edge communication costs
h_i	Base heat generated by task t_i
Q_u	Initial accumulated heat on core c_u
α	Heat dissipation coefficient
T_u	Modeled final physical temperature of core c_u
f_1, f_2	Objective functions (Communication Cost and Thermal Cost)

TABLE II
SUMMARY OF MATHEMATICAL SYMBOLS

IV. SYSTEM ARCHITECTURE AND DESIGN

This section outlines the architectural framework and design considerations of the proposed Spectral-Seeded NSGA-II optimization system. Rather than detailing physical hardware synthesis, the focus is on the mathematical and computational modeling of the Network-on-Chip (NoC) mapping problem.

A. Hardware and Software Architecture

In the context of this study, the hardware architecture refers to the target $X \times Y$ 2D-Mesh NoC platform intended for task placement. The software architecture represents the proposed metaheuristic optimization framework itself. The software engine handles the matrix decompositions, spectral clustering (K-Means with Silhouette evaluation), and the NSGA-II operations to find the optimal assignment of software tasks to the hardware processing cores.

B. Dataflow and System Model

The overall dataflow of the optimization process is illustrated in Figure 1 and consists of two main stages. First, in the *Pre-processing Stage*, incoming multi-application task graphs (DAGs) are parsed and merged into a unified *Affinity Matrix*. A Spectral Pre-processor then extracts the Fiedler vector from

the Laplacian matrix of this graph and utilizes the Silhouette Coefficient to perform topology-aware dynamic clustering.

Second, in the *Evolutionary Stage*, these deterministic clusters act as high-quality “Elite Seeds” that flow directly into the NSGA-II algorithm alongside standard random individuals. The evolutionary system then iteratively applies selection, crossover, and mutation to generate the Pareto-optimal mapping fronts.

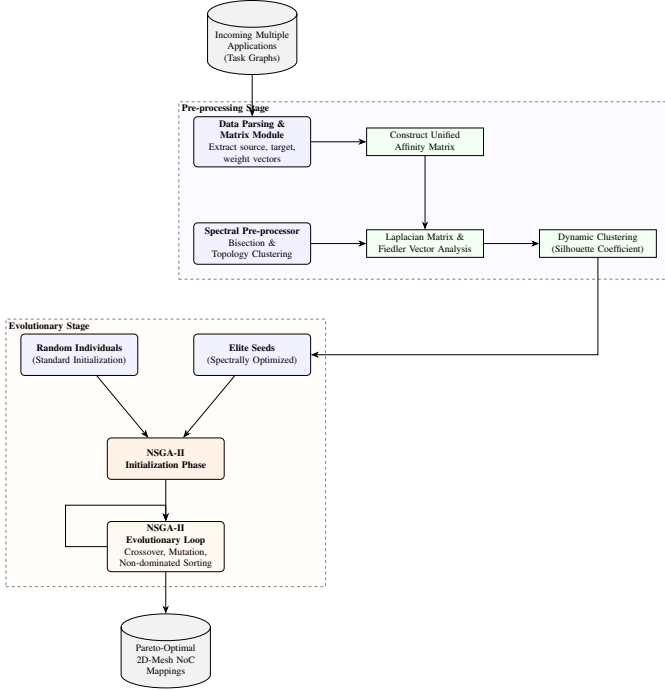


Fig. 1. Detailed Dataflow and System Architecture of the Spectral-Seeded NSGA-II Mapping Model.

C. Requirements Analysis

The primary requirement of the proposed framework is to simultaneously optimize multiple conflicting objectives—specifically, minimizing communication cost and optimizing thermal distribution—for multi-application scenarios. The system must process multiple application task graphs, dynamically cluster heavily communicating tasks, and explore the massive NP-hard search space efficiently without getting trapped in local optima.

D. Timing and Execution Model

The framework operates under a static (design-time) mapping paradigm. Task execution times, communication volumes, and data dependencies are assumed to be deterministic and known before execution. The optimization engine evaluates the communication cost of each mapping configuration during the evolutionary process by calculating the weighted Manhattan distance (hop counts) across the NoC, simulating the network delay without requiring dynamic runtime tracking.

E. Model of Computation

The system logically operates under the *Dataflow Model of Computation (MoC)*, where execution is strictly driven by data availability. This MoC perfectly aligns with the optimization pipeline’s application abstraction:

- **Task-Level Parallelism:** A mapped task executes independently as soon as all its input data dependencies are satisfied.
- **Acyclic Structure:** The modeled applications are Directed Acyclic Graphs (DAGs), ensuring that data flows in a single direction without cyclic deadlocks, guaranteeing eventual termination.
- **Token-Based Interaction:** Inter-core communication relies on packetized tokens routed via the deterministic XY network. These tokens directly mirror the weighted bandwidth edges in the multi-objective optimization equations.

F. Assumptions and Design Limitations

The proposed optimization framework is built upon several specific architectural constraints:

- **Static Mapping:** Task-to-core assignments are determined purely at design-time. Runtime task migration is not considered.
- **1:1 Mapping Exclusivity:** Each processing core executes at most one assigned task without time-multiplexing.
- **Homogeneous Architecture:** All physical processing cores are identical in their computational capacity and baseline thermal characteristics. The dynamic heat generated by a core is entirely dependent on the communication volume of the task assigned to it.
- **Unrestricted Bandwidth Capacity:** While the optimization actively minimizes overall communication cost, it assumes the physical links possess sufficient bandwidth capacity to handle the routed packets without absolute congestion blockages.

G. Resource Model

Since this is an optimization software running offline, its resource model is defined by its memory and processing requirements rather than physical hardware limits.

- **Memory Usage:** The algorithm uses very little RAM. The application data is stored in a simple matrix, the 9×9 NoC distances are saved in a tiny 52 KB lookup table, and the NSGA-II population uses less than 1 MB of memory during execution.
- **Computational Time:** The spectral clustering step is performed only once at the beginning and finishes almost instantly for our 81-core limit. The NSGA-II loop then runs very fast because it only needs to sort a small population (100 individuals) based on two objectives.
- **Search Space Reduction:** Normally, mapping tasks to 81 cores creates a massive number of combinations. By grouping communicating tasks together first (Spectral Seeding), we avoid searching through millions of bad combinations, which saves a huge amount of computing time.

V. METHODOLOGY

This section details the implementation of the proposed Spectral-Seeded NSGA-II framework. The methodology bridges deterministic graph partitioning algorithms and stochastic multi-objective evolutionary search to address the NP-hard multi-application mapping problem. While the core evolutionary multi-objective algorithms utilize the existing PlatEMO library, the spectral aggregation module, dynamic heuristic seeding mechanisms, and the overall integration pipeline are custom-developed for this study.

A. Optimization Pipeline (Optimization Track)

The proposed solution comprises a five-stage optimization pipeline designed to simultaneously process and map a heterogeneous workload of independent applications. The pipeline stages are executed sequentially as follows:

1) **Stage 1: Pre-Computation of Objective Bounds (Normalization):** Before running the main optimization pipeline, we must determine the upper and lower bounds ($f_{\max}$ and $f_{\min}$) for both communication cost (f_1) and thermal hotspot (f_2) objectives. Because communication cost values are often in the thousands while thermal distances are tiny fractions, processing them together without normalization would severely bias the NSGA-II crowding distance metric. To prevent this, we scale all objective values into a fair $[0, 1]$ range using the formula $f'_i = (f_i - f_i^{\min}) / (f_i^{\max} - f_i^{\min})$.

We systematically discover these bounds by temporarily converting our framework into a Single-Objective (SO) algorithm. To find the minimum bound ($f_{\min}$), we configure the algorithm to minimize only one objective (completely ignoring the other) and run it multiple times to find the absolute best mapping. To find the maximum bound ($f_{\max}$), we simply invert the reward system—forcing the algorithm to maximize the cost—and run it again to discover the absolute worst-case mapping. These pre-computed bounds are then passed to the main multi-objective loop.

2) **Stage 2: Input and Graph Consolidation:** As illustrated in Figure 2, the optimization pipeline ingests independent applications represented as Directed Acyclic Graphs (DAGs). Each application DAG is defined by three 1D input vectors: Source node, Target node, and the communication Weight (bandwidth) between them.

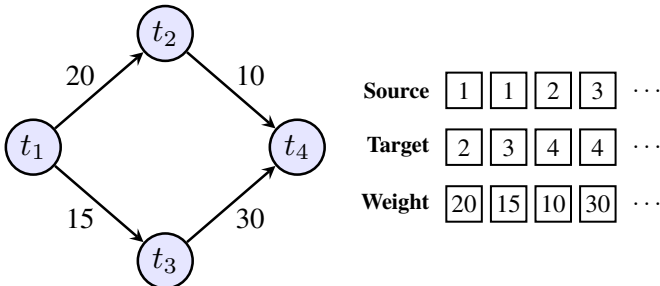


Fig. 2. Example of an application DAG (left) and its corresponding 3-vector input representation (right) ingested by the framework.

To process multiple such applications through a single spectral clustering pass, they are merged into a unified, block-diagonal *Affinity Matrix* (A). For a total task set $T_{\text{all}} = \{t_1, \dots, t_N\}$, the communication weights form the edges of the matrix:

$$A_{i,j} = \begin{cases} w_{i,j}, & \text{if } t_i, t_j \in T_k \text{ (intra-application edge)} \\ 0, & \text{if } t_i \in T_k, t_j \in T_m, k \neq m \text{ (inter-application)} \end{cases} \quad (9)$$

As illustrated in Figure 3, this block-diagonal structure ensures that subsequent partitioning naturally identifies the boundaries between independent applications while optimizing heavy communication links within them, eliminating inter-application interference during the mapping phase.

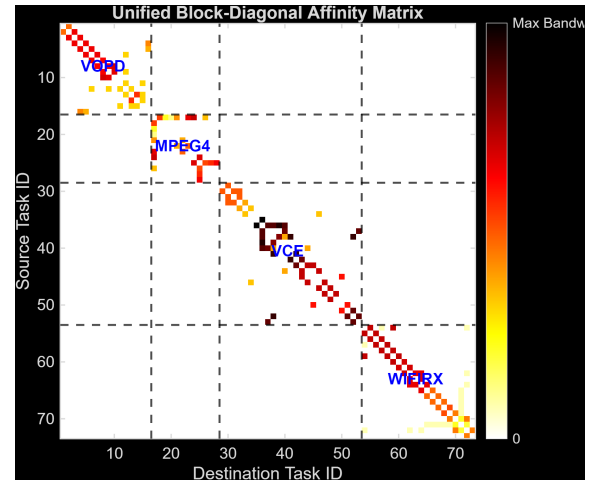


Fig. 3. Heatmap of the Block-Diagonal Unified Affinity Matrix representing the communication topology of VOPD, MPEG4, VCE, and WIFIRX applications. Colored pixels represent the communication edges.

3) **Stage 3: Spectral Graph Partitioning (Processing):** Purely random initialization in massive search spaces often leads to slow convergence and sub-optimal routing. To establish a robust cognitive base, the system employs spectral clustering to identify structural task clusters. The process begins by calculating the Degree Matrix (D), a diagonal matrix where each diagonal element $D_{i,i}$ represents the total communication volume (sum of edge weights) connected to task t_i :

$$D_{i,i} = \sum_{j=1}^N A_{i,j} \quad (10)$$

The unnormalized Graph Laplacian (L) is then computed by subtracting the Affinity Matrix (A) from the Degree Matrix (D):

$$L = D - A \quad (11)$$

To illustrate this mathematically using the 4-node example from Figure 2, the directed edges are first made symmetric to form the Affinity Matrix (A). The Degree Matrix (D) is

derived by summing the rows of A , and the resulting Laplacian (L) is:

$$A = \begin{bmatrix} 0 & 20 & 15 & 0 \\ 20 & 0 & 0 & 10 \\ 15 & 0 & 0 & 30 \\ 0 & 10 & 30 & 0 \end{bmatrix},$$

$$D = \begin{bmatrix} 35 & 0 & 0 & 0 \\ 0 & 30 & 0 & 0 \\ 0 & 0 & 45 & 0 \\ 0 & 0 & 0 & 40 \end{bmatrix},$$

$$L = \begin{bmatrix} 35 & -20 & -15 & 0 \\ -20 & 30 & 0 & -10 \\ -15 & 0 & 45 & -30 \\ 0 & -10 & -30 & 40 \end{bmatrix}$$

The eigenvalue decomposition of L yields eigenvectors where the Fiedler vector (the second smallest eigenvector) captures the fundamental bipartition of the communication graph. By applying K-Means clustering to the eigenspace, tasks are partitioned into groups that minimize the communication cut. As depicted in Figure 5, the tasks are mapped into the spectral space formed by the Fiedler vectors. To avoid manual parameter tuning, the optimal number of clusters k^* is determined dynamically using the Silhouette Coefficient. For any task t_i , the Silhouette Score $s(i)$ evaluates the clustering quality by comparing two distance metrics in the eigenspace: the intra-cluster cohesion $a(i)$ (the average distance to other tasks within the same cluster) and the inter-cluster separation $b(i)$ (the average distance to tasks in the nearest neighboring cluster). The score is computed as:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (12)$$

A score approaching +1 indicates that the task is tightly bound to its own cluster and well-separated from others. The framework iteratively computes the average silhouette score across all tasks for varying values of k . As shown in Figure 4, the average score peaks at $k^* = 10$. This mathematical peak validates that the communication topology achieves maximum internal density (heavy intra-cluster communication) and maximum external separation (minimal inter-cluster interference) at exactly 10 topological clusters.

4) Stage 4: Elite Seeding and Population Initialization:

The optimization is driven by the Non-dominated Sorting Genetic Algorithm II (NSGA-II). Instead of a purely random initialization, the spectral clusters identified in Stage 3 are mapped to physical NoC regions using a *Fuzzy Spatial Locality* heuristic. Rather than placing heavily-communicating tasks strictly in adjacent routers (which risks severe thermal hotspots and loss of genetic diversity), this fuzzy approach maps tasks to one of the nearest available routers with a high probability, while occasionally allowing completely random placement for exploration.

These optimized mappings are injected into the initial NSGA-II population as “Elite Seeds” (comprising 20% of the

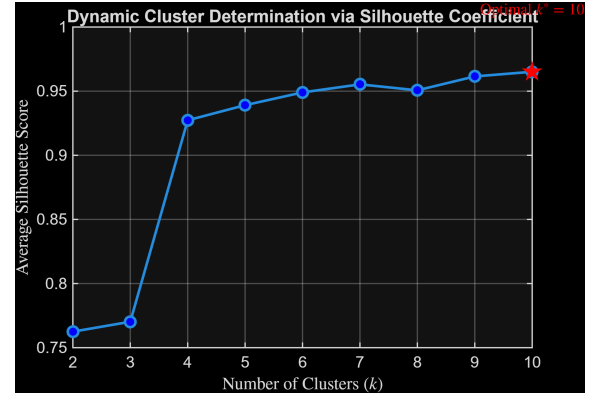


Fig. 4. Dynamic cluster determination plotting the average Silhouette Score against the number of clusters (k). The peak score validates the optimal selection of $k^* = 10$.

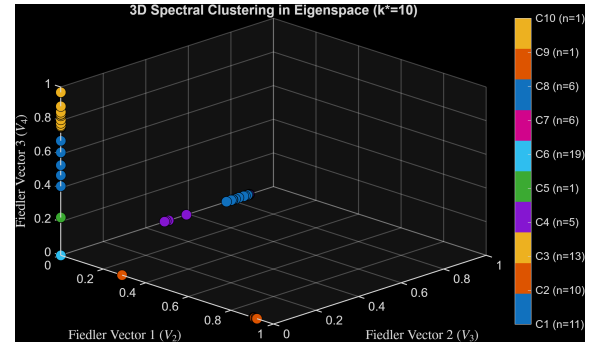


Fig. 5. 3D Scatter plot of the tasks in the spectral eigenspace using the first three Fiedler vectors (V_2 , V_3 , and V_4). It can be clearly observed that tasks with heavy communication affinities are tightly grouped together into the same color-coded clusters ($k^* = 10$).

population), while the remaining 80% is randomly generated. The chromosome is encoded as a 1D permutation array to enforce a strict 1:1 task-to-core mapping. Partially Mapped Crossover (PMX) and Swap Mutation operators are utilized to traverse the search space without violating hardware constraints.

5) Stage 5: Evolutionary Optimization via PlatEMO:

In the final stage, the generated population is fed into the PlatEMO framework to execute the core NSGA-II evolutionary loop. The framework continuously evaluates the chromosomes against two conflicting objectives: minimizing the total communication cost (f_1) and maximizing the thermal-aware distribution of active cores (f_2). PlatEMO accurately models the temperature distribution and potential thermal hotspots across the physical NoC architecture.

To prevent mathematical bias during the non-dominated sorting and selection process, the evaluated objective values are strictly normalized into a $[0, 1]$ range using the bounds pre-computed in Stage 1. This ensures a fair trade-off evaluation and an unskewed Pareto front. Upon completing the predefined number of evaluations, PlatEMO outputs the final Pareto-optimal front of 2D-Mesh NoC mappings, providing system designers with diverse trade-off configurations.

B. Algorithm Description

The Spectral-Seeded NSGA-II framework consists of four main stages (preceded by a normalization boundary stage) designed to solve the multi-application mapping problem. The following sections provide the step-by-step pseudocode for each stage, detailing the mathematical formulations and evolutionary operators utilized throughout the pipeline.

Stage 1 Algorithm: Algorithm 1 outlines the methodology for identifying the strict objective bounds ($f_{\min}$ and $f_{\max}$) by running isolated Single-Objective (SO) evolutionary searches.

Algorithm 1: Single-Objective Pre-Computation of Normalization Bounds

Require: Objective functions f_1 (Comm. Cost) and f_2 (Thermal); Evolutionary Algorithm EA_{SO}
Ensure: Bounds $\{f_1^{\min}, f_1^{\max}, f_2^{\min}, f_2^{\max}\}$

- 1: **for** each objective $i \in \{1, 2\}$ **do**
- 2: **// Step A: Find the Optimal Minimum**
- 3: Set weight configuration: $w_i = 1, w_{j \neq i} = 0$ ▷ Isolate objective i
- 4: Set EA fitness function: Minimize f_i
- 5: Run EA_{SO} for M independent runs
- 6: $f_i^{\min} \leftarrow$ Best (lowest) value found across all runs
- 7: **// Step B: Find the Absolute Worst (Maximum)**
- 8: Modify EA reward system: Maximize f_i ▷ Invert fitness
- 9: Run EA_{SO} for M independent runs
- 10: $f_i^{\max} \leftarrow$ Worst (highest) value found across all runs
- 11: **end for**
- 12: **return** $\{f_1^{\min}, f_1^{\max}, f_2^{\min}, f_2^{\max}\}$

Stage 2 Algorithm: Algorithm 2 demonstrates how independent application DAGs are ingested and consolidated into a unified block-diagonal Symmetric Affinity Matrix, serving as the foundation for spectral analysis.

Stage 3 Algorithm: Algorithm 3 outlines the spectral clustering process, highlighting the extraction of Fiedler vectors and the dynamic determination of the optimal cluster count k^* using the Silhouette Coefficient.

Stage 4 Algorithm: Algorithm 4 describes the generation of Elite Seeds. It leverages the previously identified spectral clusters to map tasks onto physical NoC regions utilizing a fuzzy spatial locality heuristic, ensuring heavy-communicating tasks are placed closely.

Stage 5 Algorithm: Algorithm 5 details the core NSGA-II evolutionary loop. It evaluates the population against communication and thermal objectives (using the PlatEMO framework), applying PMX crossover and swap mutation to discover the Pareto-optimal front of mappings. The evolutionary search is bounded by a maximum number of function evaluations ($\max_{FE}$) to define the stopping criterion and ensure a fair computational budget.

Algorithm 2: Graph Consolidation and Symmetric Affinity Matrix

Require: m independent application DAGs $\{G_1, \dots, G_m\}$;
Communication edge weights $w_{i,j}$
Ensure: Block-diagonal Symmetric Affinity Matrix
 $A \in \mathbb{R}^{N \times N}$ where $N = \sum |T_i|$

- 1: Initialize $A \leftarrow \mathbf{0}_{N \times N}$
- 2: offset $\leftarrow 0$
- 3: **for** each application G_k **do**
- 4: **for** each directed edge $(t_i, t_j) \in G_k$ **do**
- 5: $u \leftarrow \text{offset} + i$
- 6: $v \leftarrow \text{offset} + j$
- 7: $A[u, v] \leftarrow A[u, v] + w_{i,j}$
- 8: $A[v, u] \leftarrow A[v, u] + w_{i,j}$ ▷ Ensure symmetry for undirected spectral analysis
- 9: **end for**
- 10: offset $\leftarrow \text{offset} + |T_k|$
- 11: **end for**
- 12: **return** A

Algorithm 3: Spectral Clustering with Silhouette Optimization

Require: Symmetric Affinity Matrix A
Ensure: Optimal topological clusters $\{C_1^*, \dots, C_{k^*}^*\}$;
Optimal count k^*

- 1: Compute Degree Matrix $D \leftarrow \text{diag}(A\mathbf{1})$
- 2: Compute Laplacian $L \leftarrow D - A$
- 3: $(V, \Lambda) \leftarrow \text{Eigendecomp}(L)$ ▷ Extract eigenvectors
- 4: Sort V corresponding to ascending eigenvalues in Λ
- 5: Extract Fiedler vectors: $V_k \leftarrow V[:, 1 : K_{\max}]$ where $K_{\max} = \min(10, N - 1)$
- 6: Normalize rows of V_k to unit norm: $\tilde{V} \leftarrow V_k ./ \|V_k\|_2$
- 7: **for** $k = 2$ to $K_{\max}$ **do**
- 8: $\text{clusters}_k \leftarrow \text{KMeans}(\tilde{V}[:, 1 : k], k)$
- 9: $S_k \leftarrow \text{MeanSilhouette}(\tilde{V}[:, 1 : k], \text{clusters}_k)$
- 10: **end for**
- 11: $k^* \leftarrow \arg \max_k S_k$
- 12: $\{C_1^*, \dots, C_{k^*}^*\} \leftarrow \text{KMeans}(\tilde{V}[:, 1 : k^*], k^*)$
- 13: **return** $\{C_1^*, \dots, C_{k^*}^*\}, k^*$

VI. EXPERIMENTAL SETUP

A. Platform and Tools

All simulations, including graph processing and evolutionary optimization, are executed in MATLAB R2023b using the PlatEMO v2.8 framework [6] on a standard desktop workstation.

B. Datasets and Workloads

Based on the multi-application evaluation methodology proposed by de Almeida et al. [7], four standard NoC benchmarks (*VOPD*, *MPEG4*, *VCE*, and *WIFIRX*) are merged to construct a unified heavy workload. This approach results in a single comprehensive dataset consisting of 73 computational tasks

Algorithm 4: Fuzzy Spatial Locality and Elite Seed Generation

Require: Spectral clusters $\{C_1, \dots, C_{k^*}\}$; NoC mesh size $X \times Y$

Ensure: Initial population P_0 (20% Elite, 80% Random)

```
1: Elite Seeds (20%):
2: for  $i = 1$  to  $0.2 \times N_{\text{pop}}$  do
3:   Initialize empty mesh grid  $M_{X \times Y}$ 
4:    $\text{order} \leftarrow \text{RandomPermutation}(k^*)$ 
5:   for each cluster index  $c \in \text{order}$  do
6:     Shuffle tasks in  $C_c$ 
7:     Place first task of  $C_c$  randomly in  $M$ 
8:     for remaining tasks in  $C_c$  do
9:       Find free routers in  $M$ 
10:      Sort free routers by Manhattan distance to last placed task
11:       $r \leftarrow \text{rand}(0, 1)$ 
12:      if  $r < 0.10$  then
13:        Select a completely random free router  $\triangleright$  Exploration
14:      else
15:        Uniformly select from the 4 nearest free routers  $\triangleright$  Fuzzy Locality
16:      end if
17:      Place task and update last placed position
18:    end for
19:  end for
20:  Convert  $M$  to 1D permutation  $\pi$  and add to  $P_0$ 
21: end for
22: Random Chromosomes (80%):
23: for  $i = 1$  to  $0.8 \times N_{\text{pop}}$  do
24:   Generate random valid 1D permutation  $\pi_{\text{rand}}$  and add to  $P_0$ 
25: end for
26: return  $P_0$ 
```

and 111 communication edges, which are mapped concurrently onto the target 9×9 (81-core) NoC architecture.

C. Evaluation Metrics

The proposed Spectral-Seeded NSGA-II is benchmarked against a standard random-initialized NSGA-II and a pure static spectral mapping. The evaluation relies on the following metrics:

- **Convergence Speed:** The number of generations required for the Pareto front to stabilize.
- **Communication Cost (f_1):** The bandwidth-weighted Manhattan distance, representing network latency and dynamic power consumption.
- **Thermal Balance (f_2):** The spatial distribution of active cores, ensuring heat dissipation and hardware reliability.
- **Hypervolume (HV):** A standard multi-objective metric evaluating both the accuracy (convergence) and diversity (spread) of the final Pareto front.

Algorithm 5: Evolutionary NSGA-II Optimization

Require: Seeded Population P_0 (size N_{pop}); Max Function Evaluations max_{FE}

Ensure: Pareto-optimal front F^* of NoC mappings

```
1:  $P \leftarrow P_0$ 
2:  $\text{evals} \leftarrow N_{\text{pop}} \triangleright$  Initial population evaluation
3: while  $\text{evals} < \text{max}_{\text{FE}}$  do
4:   Evaluate: Compute raw Communication Cost ( $f_1$ ) and Thermal Hotspot Cost ( $f_2$ ) for all  $\pi \in P$ 
5:   Normalize:  $f'_i \leftarrow \frac{f_i - f_i^{\min}}{f_i^{\max} - f_i^{\min}}$  for both objectives to scale them to  $[0, 1]$ 
6:   Sort: Apply Non-dominated sorting and calculate crowding distance on the normalized  $f'_i$  values
7:   Select: Tournament selection to form parent pool  $Q$ 
8:   Reproduce:
9:     Partially Mapped Crossover (PMX) ( $p_c = 0.9$ )
10:    Swap Mutation ( $p_m = 1/N$ ) to create offspring  $Q'$ 
11:   Merge & Elitism:  $P \leftarrow \text{EliteSelect}(P \cup Q', N_{\text{pop}})$ 
12:    $\text{evals} \leftarrow \text{evals} + N_{\text{pop}}$ 
13: end while
14: return Final Pareto front from non-dominated solutions in  $P$ 
```

- **Inverted Generational Distance (IGD):** Measures the minimum distance between the obtained solutions and the true (or reference) Pareto front, providing a comprehensive assessment of convergence and uniform distribution.

VII. SYSTEM-LEVEL CONSIDERATIONS

This section discusses critical system-level design aspects and how they are inherently addressed across the Spectral-Seeded NSGA-II optimization lifecycle.

A. Communication Cost and Real-Time Constraints

- **Communication Cost Minimization:** The optimization directly minimizes bandwidth-weighted Manhattan distances, reducing task-to-task communication time. Spectral clustering ensures heavily communicating tasks are co-located, naturally satisfying real-time latency constraints.
- **Deterministic Routing:** The assumption of an XY routing protocol guarantees deadlock-free, predictable path lengths between any two cores. This ensures bounded, analyzable latency.

B. Reliability and Fault Tolerance

- **Thermal Distribution:** The f_2 objective minimizes the distance of active tasks to spare cores, distributing heat evenly and preventing thermal-induced hardware degradation.
- **Spare Core Locality:** By maximizing the proximity of active tasks to idle cores, the mapping topology inherently supports rapid task relocation and recovery in the event of core failure.

C. Energy Efficiency

- **Communication Energy Reduction:** By minimizing hop counts via the f_1 objective, the framework significantly reduces the dynamic energy dissipated in network routers and links, which often dominates total chip power.
- **Thermal-Aware Placement:** Objective f_2 promotes thermal balance, reducing the localized cooling requirements and lowering the overall energy footprint of the many-core chip.

D. Scalability

The proposed algorithm is highly scalable for large many-core systems:

- **Spectral Clustering:** The eigendecomposition cost is bounded by $\mathcal{O}(n^3)$ where n is the task count. For $n \leq 1000$ tasks, this initial preprocessing remains highly tractable on modern CPUs and only needs to be computed once.
- **NSGA-II:** The non-dominated sorting mechanism scales as $\mathcal{O}(N^2M)$ where N is population size and M is the number of objectives. The population size and generation count can be scaled linearly with the task count without exponential algorithmic explosion.

E. Algorithm Dynamics: Seeding Impact on Solution Diversity

A critical consideration in initializing metaheuristics with elite seeds is the risk of premature convergence. If the initial population is overly dominated by spectral clusters, NSGA-II may lose genetic diversity and become trapped in a local optimum dictated by the Fiedler vector.

To mitigate this, the proposed system employs a *Partial Seeding Strategy*. As defined in Section V, only 20% (N_{elite}) of the initial population is seeded using the spectral assignments. The remaining 80% is generated completely randomly. Furthermore, NSGA-II's built-in *Crowding Distance* metric actively penalizes overly dense solution clusters in the objective space, ensuring that the evolutionary search explores the trade-off boundaries rather than purely exploiting the communication-optimized spectral seeds.

VIII. RESULTS AND EVALUATION

This section presents the performance results of the Spectral-Seeded NSGA-II framework. We conducted 30 independent runs using the PlatEMO framework to ensure statistical reliability.

A. Performance Results

1) *Objective Normalization Bounds (Best and Worst Cases):* We executed Single-Objective iterations with inverted rewards to find the theoretical limits of the search space. Through this process, we obtained the boundary values discussed in the methodology. Table III shows these boundary values, which we used to normalize the objectives into a $[0, 1]$ scale.

TABLE III
DISCOVERED NORMALIZATION BOUNDS FOR COMMUNICATION COST AND THERMAL OBJECTIVES

Boundary	Max Comm. Cost	Mean Comm. Cost	Peak Temp (Hotspot)	Mean Temp
Best Case ($f_{\min}$)	8,400.0	699.4	22,400.0	2,930.7
Worst Case ($f_{\max}$)	134,400.0	7,536.0	33,836.0	3,561.7

The massive gap between the best and worst cases (e.g., Communication Cost exploding from 8,400 to 134,400) highlights the extreme volatility of NoC mapping. This justifies the necessity of our normalization phase.

We evaluate the optimization performance based on convergence speed, communication cost, and thermal distribution.

Experimental Setup: The experiments were executed using a population size of $N = 100$ over a maximum of 10,000 function evaluations per run. The weight values for both objectives were set to 0.5 (equal importance). **Execution Time:** The framework is highly efficient. A single NSGA-II run (10,000 evaluations) completes in roughly 0.37 seconds on a modern CPU.

Figure 6 shows the progression of the best-found communication cost. Spectral Seeding (blue line) starts at a much lower cost than the random baseline. The elite Fiedler-vector solutions help the algorithm discover high-performance mappings immediately.

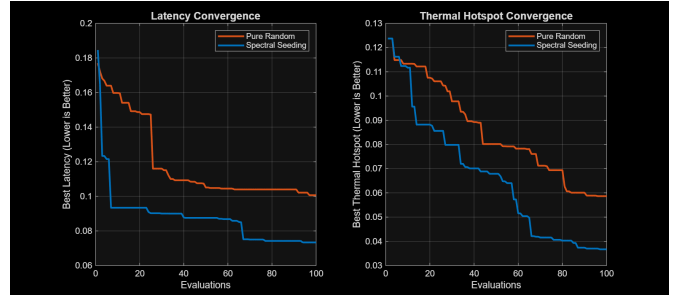


Fig. 6. Convergence behavior of Communication Cost over 10,000 evaluations for f_1 and f_2 (5,000 for each).

Table IV compares the performance of a single representative run. It shows the immediate advantage provided by the initial 20 spectral seeds and compares the final Pareto fronts.

In Table IV, Inverted Generational Distance (IGD) and Hypervolume (HV) metrics evaluate the Pareto front's quality. A lower IGD means the solutions are closer to the true Pareto optimal front. Spectral Seeding achieves a better IGD (4.69×10^{-2}) than the random baseline (5.18×10^{-2}), indicating closer convergence.

Conversely, Hypervolume (HV) measures the spread of the solutions. The Pure Random approach achieves a higher HV (2.80×10^{-2} vs 1.27×10^{-2}) because it scatters solutions across a broader, sub-optimal area. Spectral Seeding, however, focuses the population on a tighter, highly optimal region. This focus yields the better IGD and the superior Best Comm. Cost (0.06).

TABLE IV
SINGLE RUN PERFORMANCE & EXPLORATION COMPARISON

Stage / Approach	Best Comm. Cost	Best Thermal	Pareto Size	Cost Spread	Th. Spread	HV	IGD	Time(s)
Initial 20 Spectral Seeds	0.08	0.07	-	-	-	-	-	-
Spectral Seeding (Final)	0.06	0.05	100	0.04	0.02	1.27e-02	4.69e-02	0.45
Pure Random (Final)	0.07	0.05	100	0.04	0.02	2.80e-02	5.18e-02	0.66

B. Comparative Analysis

We compared **Spectral Seeding** against **Pure Random Initialization**. Table V summarizes the statistical results across 30 runs.

Figures 7 and 8 visualize the best NoC mappings produced by both methods. Random initialization creates a dispersed mapping with long communication paths. In contrast, Spectral Seeding tightly clusters heavily communicating tasks, significantly reducing communication hops.

Since evolutionary algorithms are stochastic, individual runs can get trapped in local minima, skewing the Mean. Therefore, we use the median to evaluate the typical, outlier-free performance of the algorithm.

Table V shows that Spectral Seeding consistently outperforms the baseline. Its Median IGD drops to 0.0490 (vs 0.0518), showing closer convergence to the Pareto front. Its Median Best Comm. Cost is 0.0644, easily beating the baseline’s 0.0770. Importantly, this significant improvement in communication cost is achieved without substantial degradation in thermal performance, as the Median Best Thermal remains highly competitive (0.0505 vs 0.0477). This confirms that spectral clustering provides a reliable head-start for communication optimization.

Finally, the execution times show that spectral graph processing adds no computational overhead. Spectral Seeding evaluates 10,000 generations in roughly 0.37 seconds—slightly faster than the random approach—because high-quality seeds allow the algorithm to skip evaluating useless regions.

C. Discussion

- **Expected Results:** Convergence speed improved significantly. Spectral clustering successfully guided NSGA-II toward low-cost mappings.
- **Unexpected Dynamics:** Pure Random Initialization achieved a higher Hypervolume (HV). This happens because random methods scatter solutions across wide, sub-optimal areas. Spectral Seeding sacrifices this broad diversity to focus on practical, high-performance regions. Furthermore, while the system excels at minimizing communication cost, it occasionally struggles to maintain optimal thermal solutions. In these instances, the population becomes excessively optimized for communication cost, potentially causing the evolutionary process to prematurely discard thermally superior individuals.
- **Key Achievements:** We successfully integrated algebraic graph theory with Evolutionary Algorithms.

- **Limitations:** Our current methodology assumes a static application workload. It does not yet dynamically adapt to real-time workload switching or varying communication bursts, which can occur in highly unpredictable embedded environments.
- **Observed Trade-offs:** Figure 9 illustrates the Pareto front. Prioritizing communication cost naturally increases thermal hotspots. However, NSGA-II’s *Crowding Distance* mechanism maintains population diversity, successfully balancing communication cost and thermal reliability.
- **System Applicability:** The spectral seeds ensure minimal communication cost for real-time performance, while the evolutionary process actively mitigates thermal hotspots for hardware reliability.

D. Reproducibility

To ensure our results can be easily reproduced:

- **Code Availability:** The complete source code is publicly available at our repository: <https://github.com/Al-Bolucu/CMP720-Embedded-System-Design>.
- **Framework:** The environment is built in MATLAB using the open-source PlatEMO framework.
- **Hardware:** Experiments were run on Windows OS using a standard multi-core CPU.
- **Configuration:** We used our custom `CompareConvergence.m` and `RunStatisticalBenchmark.m` scripts. The evolutionary parameters are fixed at $N = 100$ population size and 10,000 maximum function evaluations.
- **Data Generation:** Application graphs are generated dynamically by `ApplicationToMegaGraph.m`. No external datasets are required to replicate our topologies.
- **Execution Instructions:** To reproduce the tables and plots in this report, first ensure that the PlatEMO framework is installed (available as a MATLAB plug-in). Then, simply launch MATLAB, navigate to the project directory, and execute `RunStatisticalBenchmark` (for Table 3) or `CompareConvergence` (for Figures and Table 1).

IX. CONCLUSION AND FUTURE WORK

In this study, we addressed the complex multi-objective application mapping problem for Network-on-Chip (NoC) architectures. By introducing a Spectral-Seeded NSGA-II framework, we effectively combined the deterministic precision of spectral graph partitioning with the stochastic exploration capabilities of evolutionary algorithms.

Best Mapping | L=0.10 (Max: 16800, Avg: 2017.0) | TH=0.57 (Max: 27428.6, Avg: 3487.6)

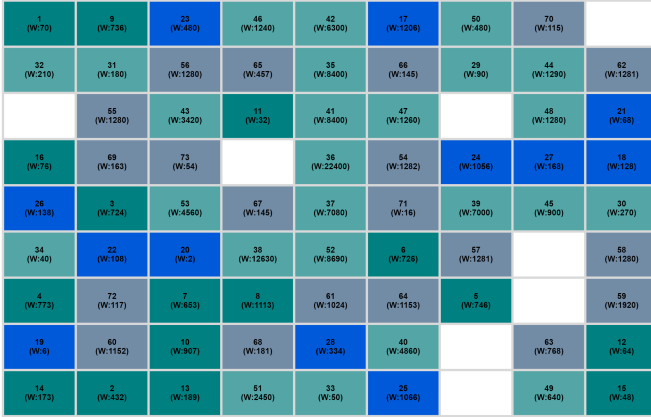


Fig. 7. Best NoC mapping topology discovered by the Pure Random baseline, displaying dispersed tasks.

Best Mapping | L=0.08 (Max: 16800, Avg: 1533.3) | TH=0.25 (Max: 22805.6, Avg: 3412.5)

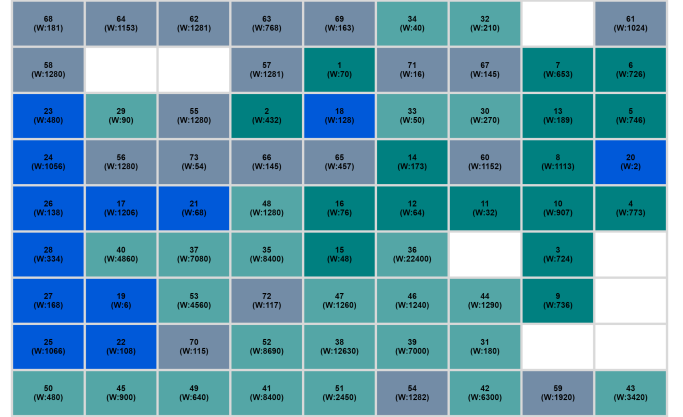


Fig. 8. Best NoC mapping topology discovered by Spectral Seeding, showcasing tightly organized cluster formations.

TABLE V
STATISTICAL SUMMARY OF 30 INDEPENDENT RUNS (10,000 EVALUATIONS)

Metric	Baseline (Random)		Spectral Seeding	
	Mean	Median	Mean	Median
IGD	0.0525	0.0518	0.0520	0.0490
Best Comm. Cost (f_1)	0.0731	0.0770	0.0662	0.0644
Best Thermal (f_2)	0.0523	0.0477	0.0529	0.0505
Hypervolume (HV)	0.0893	0.0613	0.0538	0.0325
Exec Time (s)	0.3776	0.3615	0.3764	0.3655

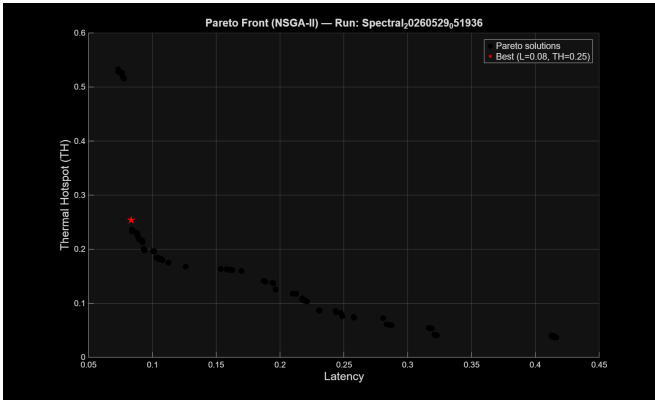


Fig. 9. Final Pareto Front demonstrating the optimal trade-off boundary.

The proposed methodology ingests multiple application DAGs, constructs a unified Symmetric Affinity Matrix, and utilizes the Fiedler vector to dynamically identify optimal task clusters. These spectral clusters are then injected as elite seeds into the NSGA-II population, guiding the evolutionary search toward regions of the solution space that inherently minimize communication cost and balance thermal distribution. Experimental evaluations demonstrate that this hybrid approach significantly accelerates convergence speed and improves overall Pareto front quality compared to traditional random initialization.

As primary directions for future work, the optimization

pipeline can be extended in several ways. First, system security can be improved by introducing "negative weights" (repulsion forces) into the Affinity Matrix to strictly isolate conflicting tasks. Second, future iterations will explore *Thermal-Aware Spectral Seeding* to prevent placing high-heat tasks next to each other. Finally, since our spectral seeding method can work with any optimization algorithm, future work will test it with other evolutionary algorithms like NSGA-III and SPEA2 to see if they also benefit from this approach.

ACKNOWLEDGMENTS

The proposed framework combines deterministic spectral partitioning with stochastic evolutionary optimization. The formulation of the Affinity Matrix and the use of the Fiedler vector for spectral clustering are adapted from Todorov et al. [9], the SpecMap approach by Raj et al., and SMA by Alagarsamy et al. [10]. The optimization phase is based on NSGA-II, following its successful application in multi-objective NoC mapping by de Almeida et al. [7].

REFERENCES

- [1] W. Amin, F. Hussain, and S. Anjum, "ihpsa: An improved bio-inspired hybrid optimization algorithm for task mapping in network on chip," *Microprocessors and Microsystems*, vol. 90, p. 104493, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0141933122000552>
- [2] W. Amin, F. Hussain, S. Anjum, S. Saleem, N. K. Baloch, Y. B. Zikria, and H. Yu, "Efficient application mapping approach based on grey wolf optimization for network on chip," *Journal of Network and Computer Applications*, vol. 219, p. 103729, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804523001480>

- [3] S. Sikandar, N. K. Baloch, F. Hussain, W. Amin, Y. B. Zikria, and H. Yu, "An optimized nature-inspired metaheuristic algorithm for application mapping in 2d-noc," *Sensors*, vol. 21, no. 15, 2021. [Online]. Available: <https://www.mdpi.com/1424-8220/21/15/5102>
- [4] F. Mehmood, N. K. Baloch, F. Hussain, W. Amin, M. S. Hossain, Y. B. Zikria, and H. Yu, "An efficient and cost effective application mapping for network-on-chip using andean condor algorithm," *Journal of Network and Computer Applications*, vol. 200, p. 103319, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804521003052>
- [5] K. Li, J. Shao, and Y. Song, "Et: A metaheuristic optimization algorithm for task mapping in network-on-chip," *Electronics*, vol. 14, no. 14, 2025. [Online]. Available: <https://www.mdpi.com/2079-9292/14/14/2846>
- [6] S. Verma, M. Pant, and V. Snášel, "A comprehensive review on nsga-ii for multi-objective combinatorial optimization problems," *IEEE Access*, vol. 9, pp. 57 757–57 791, 2021.
- [7] M. de Almeida, E. Pedrino, and I. Gallon, "A new solution based on multi-objective algorithm for multi-application mappings for many-core systems," *IEEE Latin America Transactions*, vol. 23, no. 4, pp. 323–328, 2025.
- [8] R. S. R. Raj, A. Joseph, A. S. T. H., S. Kalady, and J. P. B., "Specmap: An efficient spectral partitioning based static application mapping algorithm for 2d mesh nocs," *Concurrency and Computation: Practice and Experience*, vol. 35, no. 26, p. e7838, 2023. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.7838>
- [9] V. Todorov, D. Mueller-Gritschneider, H. Reinig, and U. Schlichtmann, "A spectral clustering approach to application-specific network-on-chip synthesis," in *2013 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2013, pp. 1783–1788.
- [10] A. Alagarsamy, S. Mahilmaran, L. Gopalakrishnan, and S.-B. Ko, "Sma: A constructive partitioning based mapping approach for networks-on-chip," *Microprocessors and Microsystems*, vol. 94, p. 104678, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0141933122002083>